

计算机网络实验 | 实验二

网络编程

小组编号: 第一组

组员姓名: 欧阳易芃、马福泉

刘昊、彭怡萱

专　　业: 计算机科学与技术

课程教师: 谢逸

起始日期: 2025 年 10 月 1 日

结束日期: 2025 年 10 月 10 日

实验地点: 实验 D402

目录

1 实验概述 **2**
1.1 实验目标 . . . 2
1.2 实验思考 . . . 2

2 实验步骤与结果 **3**
2.1 【任务一】连接实验设备 . . . 3
2.2 【任务二】编写 UDP 通信程序 . . . 3
2.2.1 学习 socket 编程 . . . 4
2.2.2 编写发送端程序 . . . 5
2.2.3 编写接收端程序 . . . 7
2.3 【任务三】运行程序并抓包分析 . . . 10
2.4 实验结果 . . . 10

3 实验思考题 **13**
3.1 程序流程图及关键函数说明 . . . 13
3.1.1 关键函数说明 . . . 14
3.2 struct sockaddr 参数详解 . . . 14
3.3 面向连接与无连接 Socket 的区别 . . . 14
3.3.1 建立 Socket 时的区别 . . . 14
3.3.2 收发数据时的区别 . . . 15
3.4 面向连接与无连接通信的比较 . . . 15
3.5 Socket 工作方式分析 . . . 15
3.6 UDP 数据包丢失原因及解决方案 . . . 16
3.7 实验过程中遇到的问题及解决方法 . . . 16
3.8 实验思路总结 . . . 16

4 拓展试验一 **17**
4.1 实验目标与原理 . . . 17
4.2 程序设计与实现 . . . 17
4.2.1 程序架构设计 . . . 17
4.2.2 服务器端核心代码 . . . 17
4.2.3 客户端核心代码 . . . 18
4.2.4 统一主程序设计 . . . 19
4.3 实验步骤与结果 . . . 20
4.3.1 实验环境配置 . . . 20
4.3.2 程序运行过程 . . . 20
4.3.3 实验结果展示 . . . 20
4.4 Wireshark 数据包分析 . . . 21
4.4.1 TCP 三次握手分析 . . . 21
4.4.2 数据传输与连接断开过程分析 . . . 21

1　实验概述

1.1　实验目标

- 连接实验设备、完成主机地址配置
- 基于套接字编程实现端到端的 UDP 通信 (1inux 或 windows)
- 发送端连续发送 100 个 UDP 数据包，在接收端统计丢失的数据包在实验时，运行 wireshark，对通信中的数据包进行跟踪分析

提示：

- 套接字编程方法可以参考网上的方法，也可以参考本文件夹中的附件。
- 编程方法不限，但要求达到实验目的。
- 可以关闭防火墙

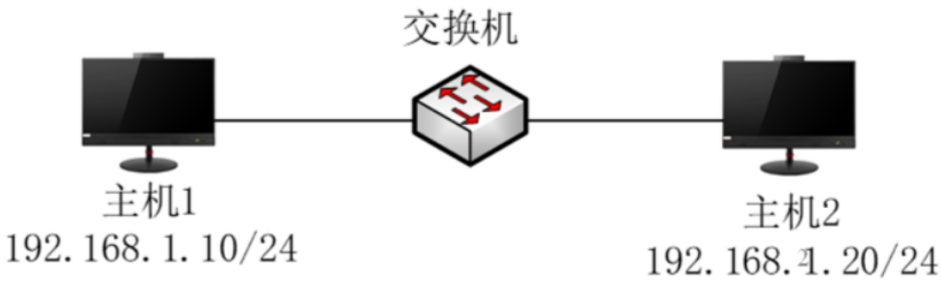


图 1: 连接示意图

1.2　实验思考

- 说明在实验过程中遇到的问题和解决方法
- 给出程序详细的流程图和对程序关键函数的详细说明。
- 使用 SocketAPI 开发通信程序中的客户端程序和服务器程序时，各需要哪些不同的函数？
- 解释 connect()、bind() 等函数中 struct sockaddr*addr 参数各个部分的含义并用具体的数据举例说明。
- 说明面向连接的客户端和面向非连接的客户端在建立 Socket 时有什么区别
- 说明面向连接的客户端和面向非连接的客户端在收发数据时有什么区别。面向非连接的客户端又是如何判断数据发送结束的？
- 比较面向连接的通信和无连接通信，它们各有什么优点和缺点？适合在哪种场合下使用？
- 实验过程中使用 Socket 时是工作在阻塞方式还是非阻塞方式？通过网络检索阐述这两种操作方式的不同。
- 注意实验时简述实验思路
- 引起 UDP 数据包丢失的原因有哪些？如何解决？

2　实验步骤与结果

2.1　【任务一】连接实验设备

根据前几次课对于交换机、路由器的熟悉，我们将主机 2 和 3 连接交换机上靠后的两个口（网线上标注了 PC 编号）：

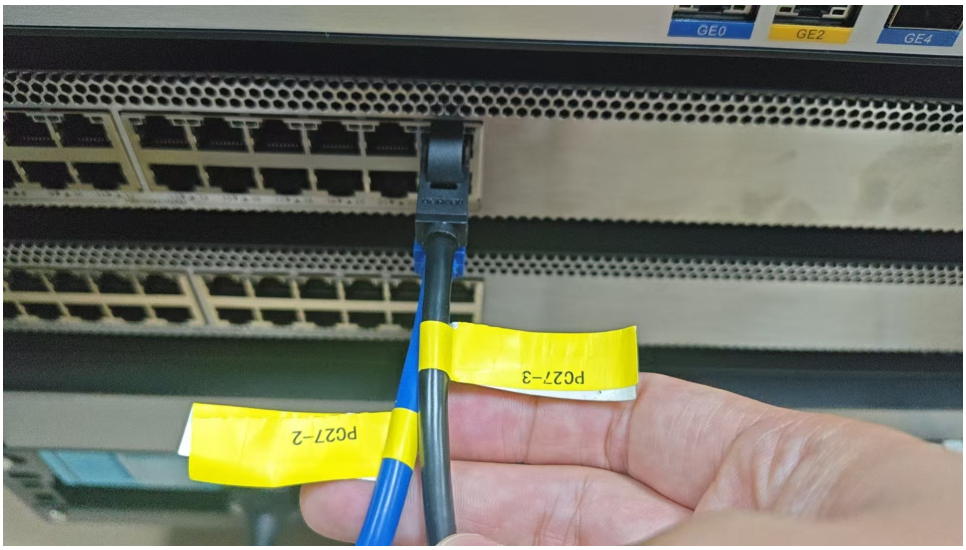


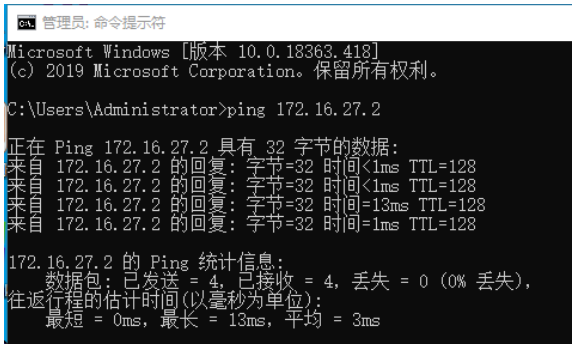
图 2: 主机 2 和 3 连接交换机

根据上次实验 ppt 中给出的 ip 地址编号规则，我们组的主机 2、3 的 ip 地址分别是 172.16.27.2 和 172.16.27.3。

打开两台电脑并分别尝试 ping 对方的 ip 地址，发现均能 ping 通，说明两台电脑已经成功连接：



(a) 2 号机 ping3 号机



(b) 3 号机 ping2 号机

图 3: 检查连接

2.2　【任务二】编写 UDP 通信程序

根据实验要求，我们应该要分别写一个发送端程序和一个接收端程序。但主要问题在于，我们之前并没有接触过 socket 编程，所以首先打开课件里的介绍和代码，进行学习。

2.2.1 学习 socket 编程

根据课件 [1] 和 csdn 的一篇详细资料 [3]，我们大概理解了 socket 作为应用层和传输层接口的作用：

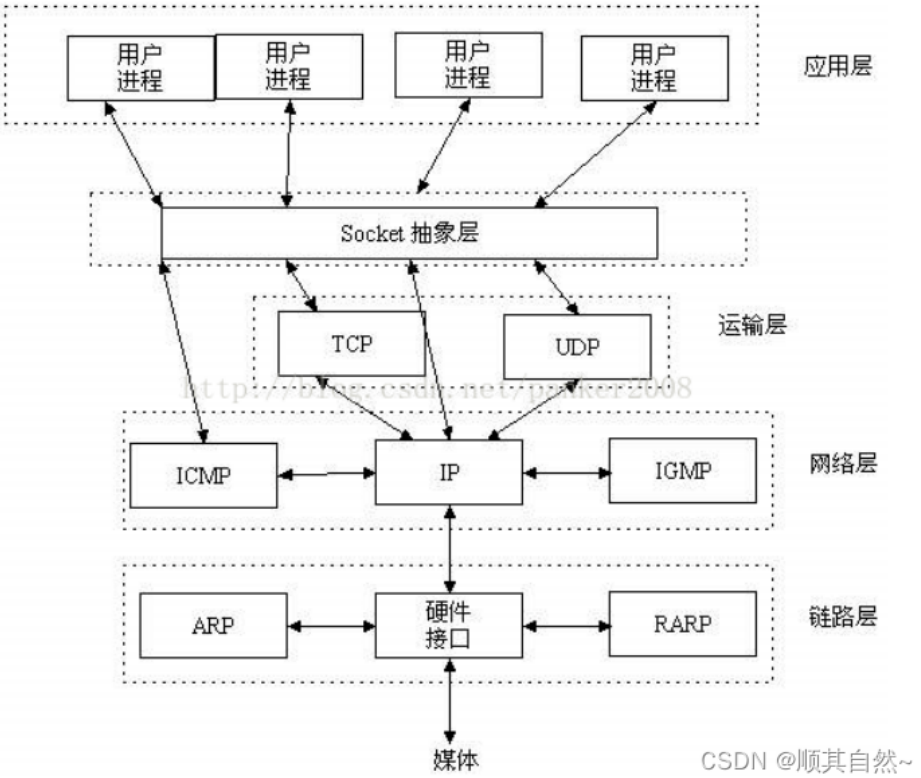


图 4: Socket 在网络协议栈中的位置

在 tcp/ip 协议中，tcp 三次握手建立连接，而 socket 也是参考 tcp 的连接方式，需要 3 次握手。在网上也找到了大神画的一个好图：

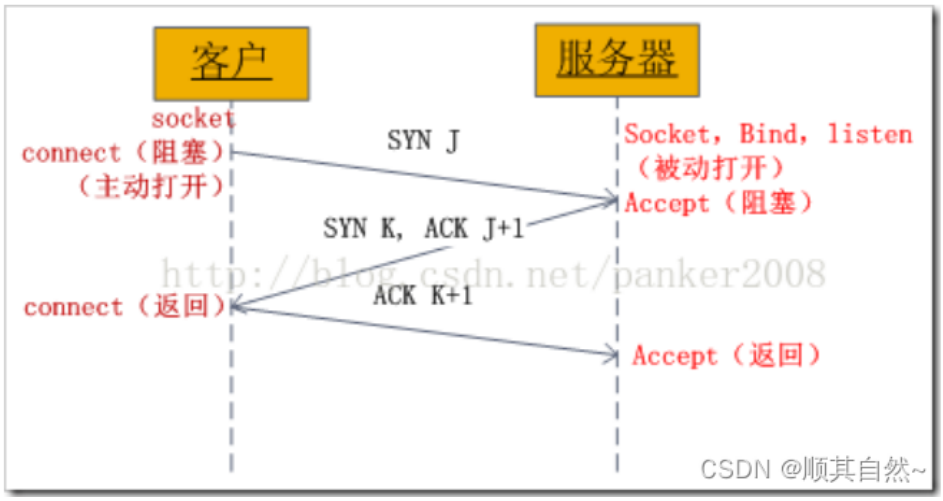


图 5: Socket 三次握手

具体过程的话，首先是通信的一方 (被动方，称为服务器) 监听某个端口，然后通信的另一方

(主动方，称为客户端) 试图对服务器端发送请求建立连接。该连接请求包含：服务器 IP 地址，服务器端口号，客户 IP 地址, 客户端口号。

由于客户端口号由客户端的系统 (TCP 进程) 自动选取一个当前未用的端口，这个四元组便可以在因特网中唯一标识一个逻辑连接。如果服务器收到客户端来的连接请求后，便发出响应建立该连接，这样就建立了一条逻辑信道连接，这样就建立了一条逻辑信道连接。

而课件中有对 Winsock 的介绍，开发过程类似文件操作，也是一个专有数据结构。而它的主要函数使用过程大概如下图：

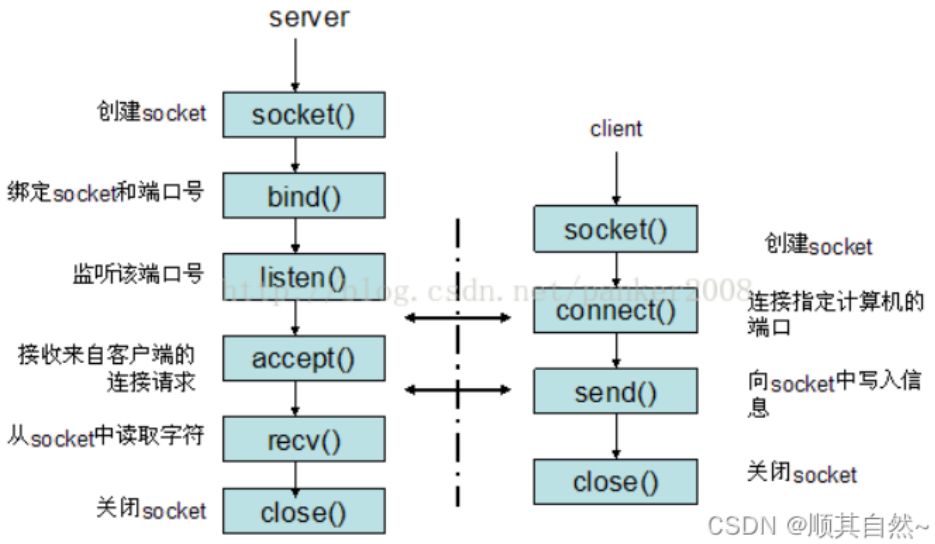


图 6: 通信过程

有了这些知识，再对照课件里面的详细函数和过程解释，我们就着手编写程序。

2.2.2　编写发送端程序

结合网络资料、课件和 ai 辅助，我们完成了程序的编写。现在拆解代码每个部分的作用：

头文件和宏定义：　在最前面，包含 winsock2，并链接 ws2_32.lib 库（这是 windows 下 socket 编程的标准库）。因为我们的发送端在 3 号机，所以 DEST_IP 设置为 2 号机的 ip。端口号可以随意设置一个不冲突的端口号，这里设置为 8888。PACKET_COUNT 表示要发送的数据包数量，SEND_DELAY 表示发送每个包之间的延迟（毫秒），可调整以模拟网络状况。

初始化 Winsock：　在定义好 WSADATA 结构和 SOCKET 等变量后，就要调用 WSAStartup() 函数初始化 Winsock 库，如果失败则输出错误并退出（后面很多地方用了相似的错误处理结构，不再赘述），这里 MAKEWORD(2,2) 表示请求使用 Winsock 2.2 版本。

创建 UDP socket：　然后我们就调用 socket() 函数创建一个 UDP 套接字，参数 AF_INET 表示 IPv4，用 SOCK_DGRAM 表示 UDP 协议。

设置目标地址：　接着我们要设置目标地址结构体 destAddr，指定目标地址族为 AF_INET，端口号为 8888（用 htons 转换为网络字节序），IP 地址为 2 号机的 ip（用 inet_addr 转换）。

发送数据包： 然后进入循环，准备发送 100 个数据包。每次循环中，我们将当前包的 ID（i）拷贝到发送缓冲区 sendBuf 的开头，这样接收端就能通过解析前 4 个字节知道这是第几个包。然后调用 sendto() 函数发送数据包，如果发送成功则打印发送的包 ID，否则打印错误信息。为了避免网络拥塞，我们在每次发送后调用 Sleep() 函数延迟 10 毫秒。

清理资源： 发送完所有包后，打印提示信息，然后关闭套接字并调用 WSACleanup() 清理 Winsock 资源。最后等待用户按键后退出程序（这一步很重要，最开始忘了加，程序运行完就直接退出了，根本看不到结果）。

```
#include <stdio.h>
#include <winsock2.h>
#include <windows.h>
#pragma comment(lib, "ws2_32.lib") // 链接Winsock库
#define DEST_IP "172.16.27.2"   // 接收端的IP地址
#define DEST_PORT 8888          // 接收端的端口号，必须与接收端程序一致
#define PACKET_COUNT 100        // 要发送的数据包数量
#define SEND_DELAY 10           // 发送每个包之间的延迟（毫秒）
int main() {
    WSADATA wsaData;
    SOCKET sendSocket;
    struct sockaddr_in destAddr;
    int i;
    char sendBuf[1024];
    int packetID;

    // 初始化Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        printf("WSAStartup failed with error %d\n", WSAGetLastError());
        return 1;
    }

    // 创建UDP socket
    sendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sendSocket == INVALID_SOCKET) {
        printf("socket() failed with error %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    // 设置目标地址（接收端）
    destAddr.sin_family = AF_INET;
    destAddr.sin_port = htons(DEST_PORT);
    destAddr.sin_addr.s_addr = inet_addr(DEST_IP); // 接收端的IP

    printf("Sender is ready. Sending %d packets to %s:%d...\n", PACKET_COUNT, DEST_IP,
     DEST_PORT);

    for (i = 1; i <= PACKET_COUNT; i++) {
        packetID = i;
        // 将包ID（一个整数）拷贝到发送缓冲区的开头
        // 这样接收端可以通过解析前4个字节来知道这是第几个包
        memcpy(sendBuf, &packetID, sizeof(packetID));

        // 发送数据包
        if (sendto(sendSocket, sendBuf, sizeof(packetID), 0, (SOCKADDR*)&destAddr,
     sizeof(destAddr)) == SOCKET_ERROR) {
```

```
            printf("sendto() failed for packet %d with error %d\n", i, WSAGetLastError()
     );
        } else {
            printf("Sent packet with ID: %d\n", i);
        }

        // 短暂延迟，避免洪水式发送导致网络或接收端处理不过来
        Sleep(SEND_DELAY);
    }

    printf("\nAll packets sent.\n");

    // 清理
    closesocket(sendSocket);
    WSACleanup();

    printf("Press any key to exit...\n");
    getchar();
    return 0;
}
```

2.2.3　编写接收端程序

这里开头跟发送端类似，但可以看出变量多了几个，主要是为了记录收到的包 ID 和统计丢包情况。而前面其他部分一直到绑定端口都类似，不再赘述。

设置接收超时时间：　这里我们调用 setsockopt() 函数设置接收超时时间为 5 秒，这样如果 5 秒内没有收到任何数据，recvfrom() 函数就会返回，避免程序永远阻塞。不过其实我们也发现 5 秒有点短了，经常两边机子会没准备好，但如果太长又会等很久，所以这里先保持设置 5 秒。

接收数据包：　然后进入一个无限循环，调用 recvfrom() 函数接收数据包。如果 recvfrom() 返回 SOCKET_ERROR，我们检查错误码，如果是 WSAETIMEDOUT，说明 5 秒内没有收到数据，我们打印提示并跳出循环结束接收；否则打印错误信息继续等待下一个包。

如果成功接收到数据，我们检查接收长度是否至少包含一个整数（4 字节），然后从缓冲区前 4 个字节提取包 ID，并标记该包已收到。这里我们用一个数组 receivedPackets 来记录收到的包，索引对应包 ID，值为 1 表示收到，0 表示未收到。

统计丢包情况：　当退出接收循环后，我们遍历 receivedPackets 数组，统计哪些包未收到，并打印丢失的包 ID，最后计算并打印总的丢包数和丢包率。

```
#include <stdio.h>
#include <winsock2.h>
#include <windows.h>

#pragma comment(lib, "ws2_32.lib") // 链接Winsock库

#define RECV_PORT 8888 // 接收端监听的端口号，可以自行修改
#define PACKET_COUNT 100 // 期望接收的数据包总数

int main() {
    WSADATA wsaData;
    SOCKET recvSocket;
```

```
    struct sockaddr_in recvAddr, senderAddr;
    int senderAddrSize = sizeof(senderAddr);
    int recvLen;
    char recvBuf[1024];
    int receivedPackets[PACKET_COUNT + 1] = {0}; // 数组用于标记收到的包，索引从1到100
    int i, lostCount = 0;
    int packetID;

    // 初始化Winsock
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        printf("WSAStartup failed with error %d\n", WSAGetLastError());
        return 1;
    }

    // 创建UDP socket
    recvSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (recvSocket == INVALID_SOCKET) {
        printf("socket() failed with error %d\n", WSAGetLastError());
        WSACleanup();
        return 1;
    }

    // 绑定接收端的地址和端口
    recvAddr.sin_family = AF_INET;
    recvAddr.sin_port = htons(RECV_PORT);
    recvAddr.sin_addr.s_addr = htonl(INADDR_ANY); // 监听所有网络接口

    if (bind(recvSocket, (SOCKADDR*)&recvAddr, sizeof(recvAddr)) == SOCKET_ERROR) {
        printf("bind() failed with error %d\n", WSAGetLastError());
        closesocket(recvSocket);
        WSACleanup();
        return 1;
    }

    printf("Receiver is running on 172.16.27.2:%d...\nWaiting for packets...\n",
     RECV_PORT);

    // 设置超时时间（5秒）
    // 如果5秒内没有收到任何数据，recvfrom将返回，避免程序永远阻塞
    DWORD timeout = 5000;
    setsockopt(recvSocket, SOL_SOCKET, SO_RCVTIMEO, (char*)&timeout, sizeof(timeout));

    // 接收数据包
    while (1) {
        recvLen = recvfrom(recvSocket, recvBuf, sizeof(recvBuf), 0, (SOCKADDR*)&
     senderAddr, &senderAddrSize);
        if (recvLen == SOCKET_ERROR) {
            int error = WSAGetLastError();
            if (error == WSAETIMEDOUT) {
                printf("\nNo data received for 5 seconds. Stopping reception.\n");
                break;
            } else {
                printf("recvfrom() failed with error %d\n", error);
                continue;
            }
        }

        // 确保接收到的数据足以包含一个整数（我们的包ID）
        if (recvLen >= sizeof(int)) {
```

```
            // 从接收缓冲区的前4个字节中提取包ID
            packetID = *((int*)recvBuf);
            if (packetID >= 1 && packetID <= PACKET_COUNT) {
                receivedPackets[packetID] = 1; // 标记这个包已收到
                printf("Received packet with ID: %d\n", packetID);
            }
        } else {
            printf("Received a packet with invalid length.\n");
        }
    }

    // 统计丢失的包
    for (i = 1; i <= PACKET_COUNT; i++) {
        if (receivedPackets[i] == 0) {
            lostCount++;
            printf("Packet %d was lost.\n", i);
        }
    }

    // 输出结果
    printf("\n===== Statistics =====\n");
    printf("Expected packets: %d\n", PACKET_COUNT);
    printf("Received packets: %d\n", PACKET_COUNT - lostCount);
    printf("Lost packets: %d\n", lostCount);
    printf("Packet loss rate: %.2f%%\n", (lostCount * 100.0) / PACKET_COUNT);

    // 清理
    closesocket(recvSocket);
    WSACleanup();

    printf("Press any key to exit...\n");
    getchar();
    return 0;
}
```

完成程序后，我就在 vscode 上面按编译，结果出现了报错：

```
正在启动生成...
cmd /c chcp 65001>nul && "C:\Program Files\mingw64\bin\gcc.exe" -fdiagnostics-color=
     always -g C:\Users\18044\Desktop\down\sender.c -o C:\Users\18044\Desktop\down\
     sender.exe
C:\Users\18044\AppData\Local\Temp\ccLVTdc9.o: In function ‘main’:
C:/Users/18044/Desktop/down/sender.c:21: undefined reference to ‘__imp_WSAStartup’
C:/Users/18044/Desktop/down/sender.c:22: undefined reference to ‘__imp_WSAGetLastError’
C:/Users/18044/Desktop/down/sender.c:27: undefined reference to ‘__imp_socket’
C:/Users/18044/Desktop/down/sender.c:29: undefined reference to ‘__imp_WSAGetLastError’
C:/Users/18044/Desktop/down/sender.c:30: undefined reference to ‘__imp_WSACleanup’
C:/Users/18044/Desktop/down/sender.c:36: undefined reference to ‘__imp_htons’
C:/Users/18044/Desktop/down/sender.c:37: undefined reference to ‘__imp_inet_addr’
C:/Users/18044/Desktop/down/sender.c:48: undefined reference to ‘__imp_sendto’
C:/Users/18044/Desktop/down/sender.c:49: undefined reference to ‘__imp_WSAGetLastError’
C:/Users/18044/Desktop/down/sender.c:61: undefined reference to ‘__imp_closesocket’
C:/Users/18044/Desktop/down/sender.c:62: undefined reference to ‘__imp_WSACleanup’
collect2.exe: error: ld returned 1 exit status
```

查询发现，这是因为 gcc 编译器没有自动链接 ws2_32.lib 库，所以我们需要手动指定链接这个库。解决方法是在编译命令中添加-lws2_32 参数：

```
gcc -fdiagnostics-color=always -g C:\Users\18044\Desktop\down\sender.c -o C:\Users
    \18044\Desktop\down\sender.exe -lws2_32
```

2.3 【任务三】运行程序并抓包分析

编译成 exe 之后，我们就把两个程序分别放到两台电脑上运行。这里我们让 3 号机运行发送端程序，2 号机运行接收端程序。同时，我们在两台电脑上都打开 wireshark 进行抓包分析，条件直接设置的是来自于对方 ip 的包。

Step1: 打开 sender 与 receiver 两台电脑的 wireshark，并提前设置好过滤内容：

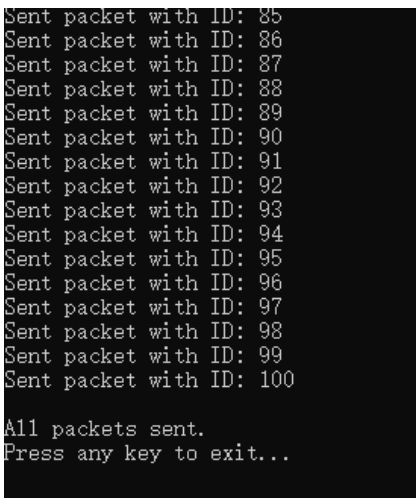
- **sender 端(IP=172.16.27.2)：在过滤器中设置 ip dst == 172.16.27.2,只查看接受端 ip=172.16.27.2 的包。**
- **receiver 端(IP=172.16.27.3)：在过滤器中设置 ip src == 172.16.27.3,只查看发送端 ip=172.16.27.3 的包。**

Step2: 先提前打开 receiver 端的接收程序，再打开 sender 端的发送程序，进行数据的发送与接收。注意：由于丢包时间设置的是 **5s**，所以需提前打开 receiver 端的接收程序，避免因等待时间过长而丢包。

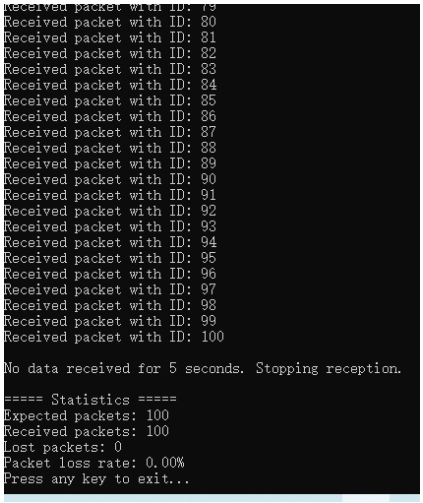
Step3: 等待数据传输完成后，停止抓包，并保存抓包结果。

2.4 实验结果

从上面的代码可以看见，我们在程序中加上了输出，所以可以直接看到发送端和接收端的运行结果。在过程中出现了一些意外情况，首先是因为我们分 2 人操控 2 台机子，有时候 5 秒的时间到了，接收端还没来得及收到数据包，所以会提示超时并结束接收。其次是有时候会出现丢包现象，可能是网络状况不佳或者发送太快导致的。我们尝试调整了发送延迟时间，最终将发送延迟调整为 10ms，基本上就没有丢包了。程序运行截图如下所示：



(a) 发送端运行结果



(b) 接收端运行结果

图 7: 发送端和接收端运行结果截图

从两张图可以看出，发送端成功发送了 100 个数据包，而接收端也成功接收了 100 个数据包，

没有出现丢包现象。

同时，我们也保存了 wireshark 的抓包结果，分别如下所示：

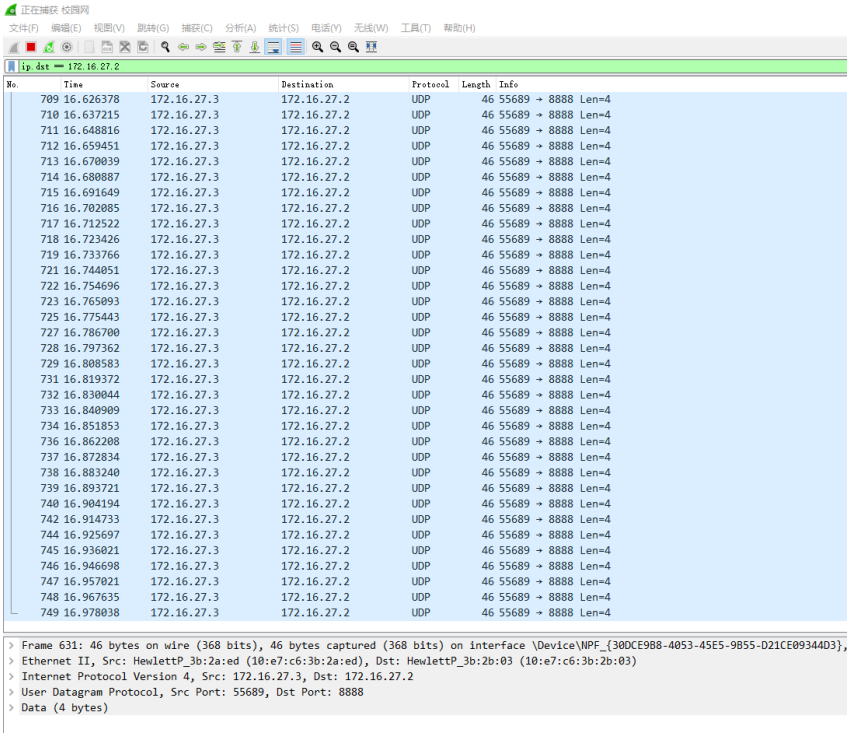


图 8: sender 端抓包结果

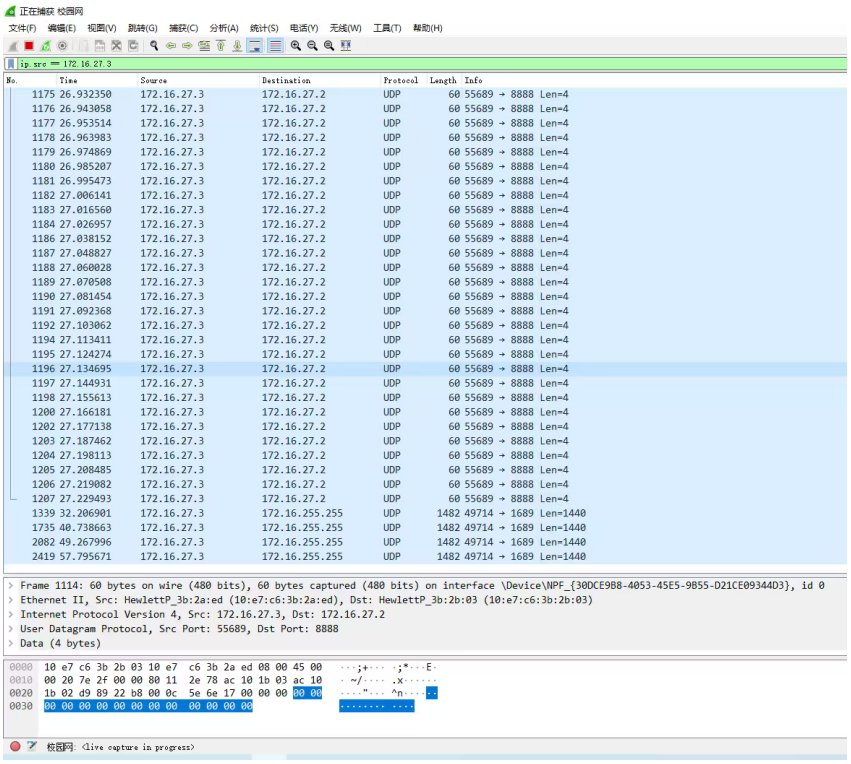


图 9: receiver 端抓包结果

这里 wireshark 抓包结果显示，发送端发送的 UDP 包确实到达了接收端，且包的源 IP、目的

IP 和端口号均与我们程序中设置的一致。每个包的长度为 8 字节（4 字节包 ID + 4 字节其他数据），符合预期。

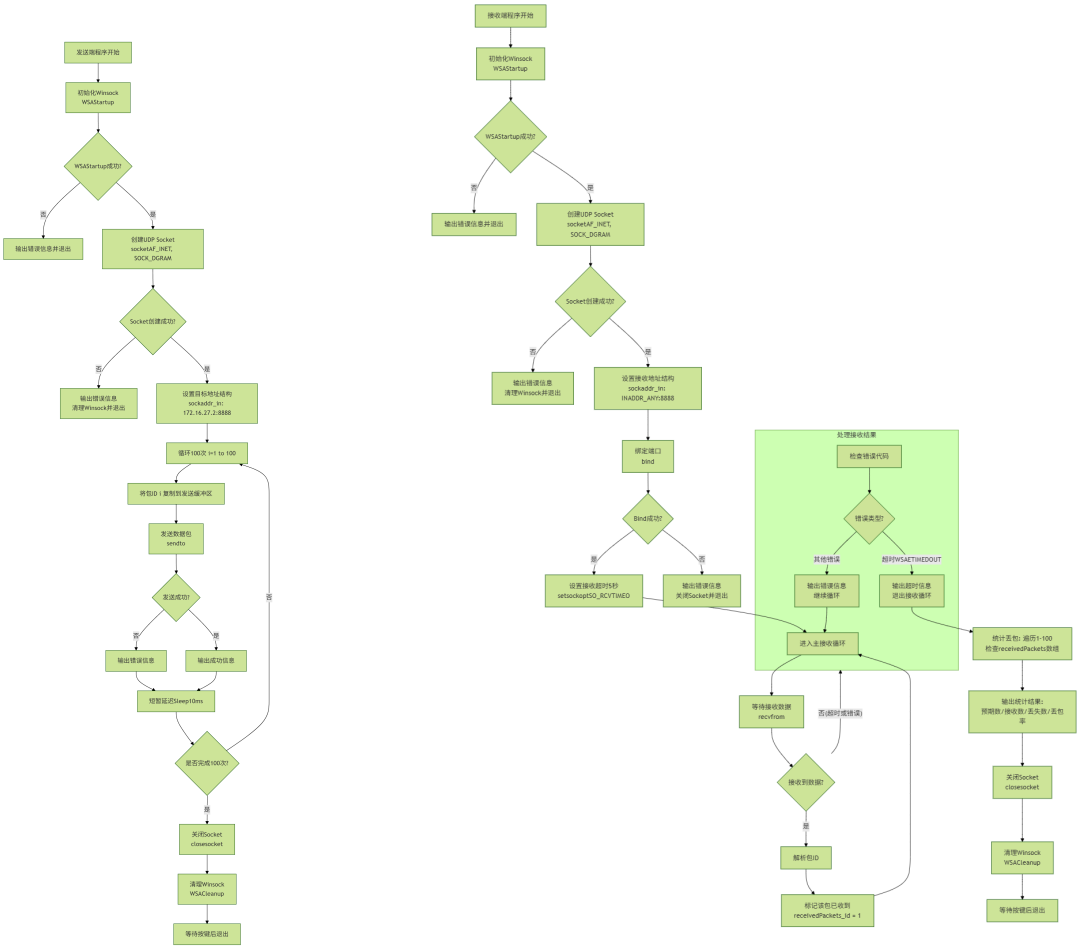
实验结果总结：

本次实验通过在 sender 和 receiver 两台主机上分别使用 Wireshark 抓包，成功验证了数据包的发送与接收过程。实验结果显示，发送端成功发送了 100 个数据包，接收端也全部正确接收，未出现丢包现象。抓包结果进一步证明了数据包的源 IP、目的 IP 及端口号均与实验设置一致，数据传输过程稳定可靠，达到了预期实验目标。这说明本次网络通信程序设计正确，数据传输过程中的各项参数设置合理，能够满足实际应用需求。

3　实验思考题

3.1　程序流程图及关键函数说明

根据程序的编写逻辑，我们绘制了发送端和接收端的程序流程图如下所示（这里用的 mermaid flow chart 网站在线生图，好像有点小了，可以放大来看）：



(a) 发送端运行流程图　　(b) 接收端运行流程图

图 10: 发送端和接收端程序流程图

而流程实际上并不复杂，在前面放的那张示意图（图 6）已经很清晰了，这里我们再用文字简单总结一下：

发送端程序流程：

1. 初始化 *Winsock* 库
2. 创建 *UDP* 套接字
3. 设置目标地址结构
4. 循环发送 *100* 个数据包
5. 清理资源并退出

接收端程序流程：

1. 初始化 *Winsock* 库
2. 创建 *UDP* 套接字
3. 绑定本地地址和端口
4. 设置接收超时时间
5. 循环接收数据包并记录
6. 统计丢包情况并输出结果
7. 清理资源并退出

3.1.1 关键函数说明

前面有提到过，这里就再简单总结一下程序中用到的关键函数及其作用：

- **WSAStartup()：初始化 Winsock 库，必须在使用任何 socket 函数之前调用**
- **socket()：创建套接字，参数 AF_INET 表示 IPv4，SOCK_DGRAM 表示 UDP 协议**
- **bind()：仅接收端调用，将套接字绑定到本地地址和端口**
- **sendto()：发送端用于发送数据包，包内容前 4 字节为包 ID（int 类型）**
- **recvfrom()：接收端用于接收数据包，解析前 4 字节获得包 ID**
- **setsockopt()：接收端设置 SO_RCVTIMEO，超时时间为 5 秒，避免阻塞**
- **Sleep()：发送端每发送一个包后延迟 10ms，避免网络拥塞**
- **统计丢包：接收端用 receivedPackets 数组标记收到的包，统计丢失包数量和丢包率**

3.2 struct sockaddr 参数详解

struct sockaddr_in 结构体各部分含义：

- *sin_family: 地址族，AF_INET 表示 IPv4*
- *sin_port: 端口号，发送端和接收端均为 8888，需用 htons() 转换*
- *sin_addr.s_addr: IP 地址，发送端目标为"172.16.27.2"，接收端为 INADDR_ANY*

具体示例：

```
// 发送端
destAddr.sin_family = AF_INET;
destAddr.sin_port = htons(8888);
destAddr.sin_addr.s_addr = inet_addr("172.16.27.2");

// 接收端
recvAddr.sin_family = AF_INET;
recvAddr.sin_port = htons(8888);
recvAddr.sin_addr.s_addr = htonl(INADDR_ANY);
```

3.3 面向连接与无连接 Socket 的区别

3.3.1 建立 Socket 时的区别

表 1: 建立 Socket 时的区别对比

特性	面向连接（TCP）	无连接（UDP）
创建套接字	socket(AF_INET, SOCK_STREAM, IPPROTO_TCP)	socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)
客户端连接	需要调用 connect() 建立连接	不需要建立连接，直接发送数据
服务器端等待	需要调用 listen() 和 accept() 等待连接	只需 bind() 绑定本地地址
连接状态	需要维护连接状态	无连接状态
资源开销	较高（连接维护）	较低（无连接维护）

3.3.2 收发数据时的区别

表 2: 收发数据时的区别对比

特性	面向连接（TCP）	无连接（UDP）
发送函数	send()	sendto()，每包前 4 字节为包 ID
接收函数	recv()	recvfrom()，解析包 ID
地址指定	连接建立后不需要指定目标地址	每次发送都需指定目标地址
数据传输方式	数据流式传输，保证顺序和完整性	数据报式传输，不保证顺序和到达性
结束判断	通过连接关闭或特殊标记	接收端超时（5 秒无数据）自动结束
可靠性	高（自动重传、确认）	低（无重传机制）

3.4 面向连接与无连接通信的比较

表 3: 面向连接与无连接通信特性对比

通信方式	优点	缺点	适用场合
TCP	可靠性高，保证数据完整性和顺序	开销大，需要维护连接状态	文件传输
	提供流量控制和拥塞控制	传输效率相对较低	网页浏览
	自动重传丢失的数据	不适合实时性要求高的应用	电子邮件
UDP	传输效率高，开销小	不保证数据可靠性	实时音视频传输
	实时性好，延迟低	可能出现数据丢失、重复、乱序	DNS 查询
	简单快速，无连接状态	无流量控制机制	在线游戏

3.5 Socket 工作方式分析

本实验中 Socket 工作在**阻塞方式**。

阻塞方式：

- **recvfrom() 函数会一直等待直到收到数据或超时**
- **程序执行流会被挂起，等待 I/O 操作完成**
- **简单易用，但可能导致程序响应性差**

非阻塞方式：

- **I/O 操作立即返回，不会阻塞程序执行**
- **需要轮询或使用 select/epoll 等机制检查 I/O 状态**
- **程序响应性好，但编程复杂度高**

本实验通过设置 SO_RCVTIMEO 选项实现了带超时的阻塞接收，在指定时间内没有数据时自动返回。

3.6 UDP 数据包丢失原因及解决方案

UDP 数据包丢失的主要原因：

- 网络拥塞：网络带宽不足或路由器缓冲区溢出
- 接收缓冲区溢出：接收端处理速度跟不上发送速度
- 网络设备故障：交换机、路由器等设备出现问题
- 传输错误：物理链路传输过程中出现比特错误
- 发送速度过快：发送端发送频率超过网络承载能力

解决方案：

- 应用层重传：实现 ***ACK*** 确认机制，超时重传丢失的包
- 流量控制：动态调整发送速率，避免网络拥塞
- 增大缓冲区：适当增大接收端缓冲区大小
- 数据包序号：添加序列号检测丢包和重复
- 错误检测与纠正：使用校验和、前向纠错等技术
- 冗余传输：关键数据多次发送或使用不同路径

3.7 实验过程中遇到的问题及解决方法

遇到的主要问题：

1. **IP 地址配置错误**: 初期设置的 IP 地址不在同一网段，导致无法通信
   - **解决方法：重新配置 IP 地址为 172.16.27.2 和 172.16.27.3，确保在同一网段**
2. **防火墙阻止**: Windows 防火墙阻止程序网络访问
   - **解决方法：在防火墙中添加程序例外或临时关闭防火墙**
3. **端口占用**: 8888 端口被其他程序占用
   - **解决方法：使用 netstat 命令检查端口占用情况，更换端口或终止占用程序**
4. **接收超时设置**: 初期未设置接收超时，程序可能无限等待
   - **解决方法：使用 setsockopt() 设置 SO_RCVTIMEO 选项，避免程序永久阻塞**

3.8 实验思路总结

本实验的核心思路：

1. 设计发送端程序，连续发送 100 个带有序列号的 UDP 数据包
2. 设计接收端程序，记录收到的数据包序列号并统计丢包情况
3. 使用 Wireshark 抓包工具监控网络传输过程
4. 通过对比发送和接收的数据包数量，分析网络传输的可靠性
5. 深入理解 UDP 协议的特点和 Socket 编程的基本原理

通过本实验，我们成功验证了 UDP 通信的基本原理，掌握了 Socket 编程的核心技术，并学会了使用网络分析工具进行数据包跟踪分析。实验结果表明，在良好的网络环境下，UDP 传输是可靠的，但在实际应用中需要考虑各种可能导致丢包的因素。

4　拓展试验一

4.1　实验目标与原理

本拓展实验旨在深入理解 TCP 协议的工作原理，通过基于 TCP 套接字的数据通信实现多客户端聊天室功能。

实验核心原理：

- 使用 ***TCP*** 协议建立可靠的客户端-服务器连接
- 服务器支持多客户端同时连接，实现聊天室功能
- 通过线程技术处理并发连接和消息广播
- 使用 ***Wireshark*** 分析 ***TCP*** 三次握手和数据传输过程

TCP 协议作为面向连接的可靠传输协议，与 UDP 的主要区别在于需要建立连接、保证数据顺序和完整性。本实验通过实现聊天室功能，深入体验 TCP 编程的特点。

4.2　程序设计与实现

4.2.1　程序架构设计

基于 TCP 的聊天室系统采用客户端-服务器架构，服务器端维护多个客户端连接，实现消息的接收和广播功能。

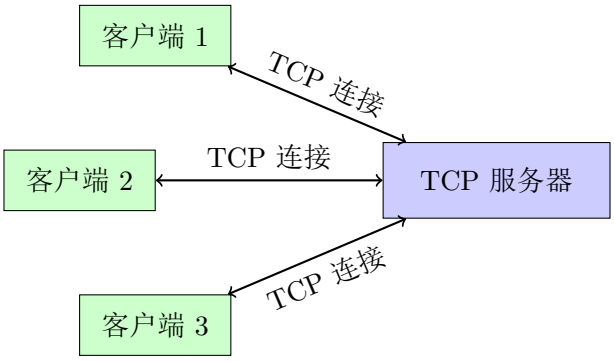


图 11: TCP 聊天室系统架构图

4.2.2　服务器端核心代码

服务器端的核心在于客户端管理和消息广播机制。我们使用线程池技术处理多客户端并发连接：

```
1  // 客户端信息结构
2  typedef struct {
3      SOCKET socket;
4      struct sockaddr_in addr;
```

```
    int id;
    char username[50];
    int active;
} ClientInfo;

// 全局变量
ClientInfo clients[MAX_CLIENTS];
int clientCount = 0;
CRITICAL_SECTION clientsLock;

// 主循环：接受客户端连接
while (1) {
    clientSocket = accept(serverSocket, (SOCKADDR*)&clientAddr, &clientAddrSize);
    if (clientSocket == INVALID_SOCKET) {
        printf("accept() failed with error %d\n", WSAGetLastError());
        continue;
    }

    // 添加新客户端
    int clientID = addClient(clientSocket, clientAddr);
    if (clientID == -1) {
        printf("服务器已满，拒绝连接 %s:%d\n",
               inet_ntoa(clientAddr.sin_addr), ntohs(clientAddr.sin_port));
        closesocket(clientSocket);
        continue;
    }

    // 为每个客户端创建处理线程
    HANDLE hThread = (HANDLE)_beginthreadex(NULL, 0, handleClient, &clients[clientID],
     0, NULL);
    // ...existing code...
}

// 广播消息给所有客户端
void broadcastMessage(char* message, int senderID) {
    EnterCriticalSection(&clientsLock);

    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].active && i != senderID) {
            if (send(clients[i].socket, message, strlen(message), 0) == SOCKET_ERROR) {
                printf("发送消息到客户端 %d 失败\n", clients[i].id);
            }
        }
    }

    LeaveCriticalSection(&clientsLock);
}
```

4.2.3　客户端核心代码

客户端程序相对简单，主要负责建立 TCP 连接和双向数据通信：

```
// 连接到服务器
if (connect(clientSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
    printf("连接服务器失败，错误代码: %d\n", WSAGetLastError());
    printf("请确保服务器正在运行并且IP地址正确\n");
    closesocket(clientSocket);
```

```
    WSACleanup();
    return 1;
}

printf("成功连接到服务器! \n\n");

// 创建接收消息的线程
HANDLE hReceiveThread = (HANDLE)_beginthreadex(NULL, 0, receiveMessages, NULL, 0, NULL);

// 主循环: 发送消息
while (running) {
    printf("输入消息 (输入 'quit' 退出): ");
    if (fgets(sendBuffer, sizeof(sendBuffer), stdin) != NULL) {
        // 移除换行符
        char* newline = strchr(sendBuffer, '\n');
        if (newline) *newline = '\0';

        // 检查是否要退出
        if (strcmp(sendBuffer, "quit") == 0) {
            running = 0;
            break;
        }

        // 发送消息到服务器
        if (send(clientSocket, sendBuffer, strlen(sendBuffer), 0) == SOCKET_ERROR) {
            printf("发送消息失败, 错误代码: %d\n", WSAGetLastError());
            running = 0;
            break;
        }
    }
}
```

4.2.4 统一主程序设计

为了便于测试和演示，我们设计了统一的主程序，可以选择运行服务器或客户端模式：

```
int main() {
    int choice;

    printf("============================================================\n");
    printf("            TCP套接字数据通信实验程序\n");
    printf("============================================================\n");
    printf("实验功能:\n");
    printf("1. 基于TCP协议的客户端-服务器通信\n");
    printf("2. 支持多客户端同时连接\n");
    printf("3. 实现聊天室功能和数据交互\n");
    printf("4. 可用Wireshark分析网络数据包\n");
    printf("============================================================\n\n");

    printf("请选择运行模式:\n");
    printf("1. 运行TCP服务器\n");
    printf("2. 运行TCP客户端\n");
    printf("3. 退出程序\n");
    printf("\n请输入选择 (1-3): ");

    scanf("%d", &choice);

```

```
    switch (choice) {
        case 1: runServer(); break;
        case 2: runClient(); break;
        case 3: printf("\n程序退出。\n"); break;
        default: printf("\n无效选择，程序退出。\n"); break;
    }

    return 0;
}
```

4.3 实验步骤与结果

4.3.1 实验环境配置

按照主实验的网络配置，我们继续使用 2 号机（172.16.27.2）作为服务器，3 号机（172.16.27.3）作为客户端进行测试。

4.3.2 程序运行过程

Step 1: 编译程序并启动 Wireshark 监控

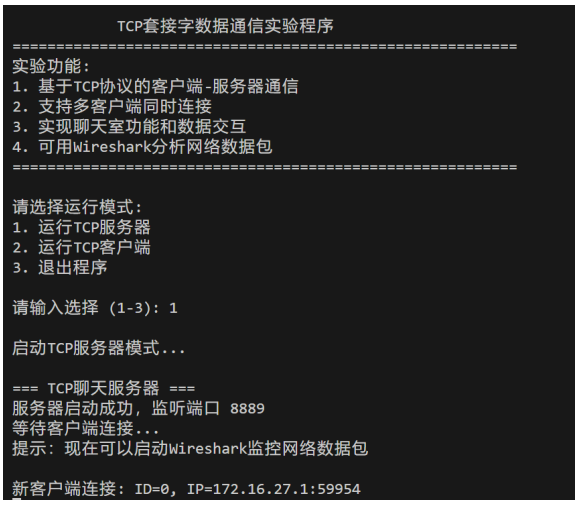
```
gcc -o extend.exe extend.c -lws2_32
```

Step 2: 在 2 号机上启动服务器程序，选择模式 1

Step 3: 在 3 号机上启动客户端程序，选择模式 2

Step 4: 进行聊天测试并观察数据包传输

4.3.3 实验结果展示

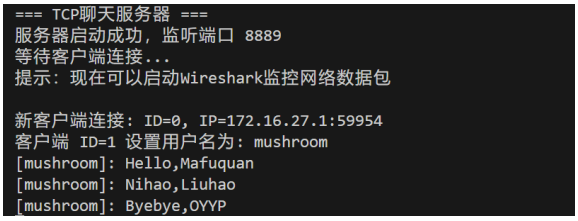


(a) TCP 服务器运行界面

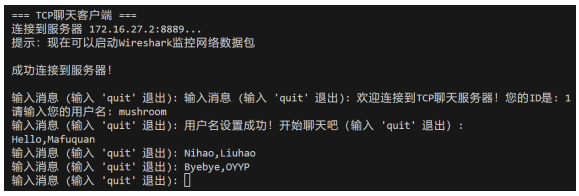


(b) TCP 客户端运行界面

图 12: TCP 聊天室程序运行截图



(a) 服务器端聊天演示



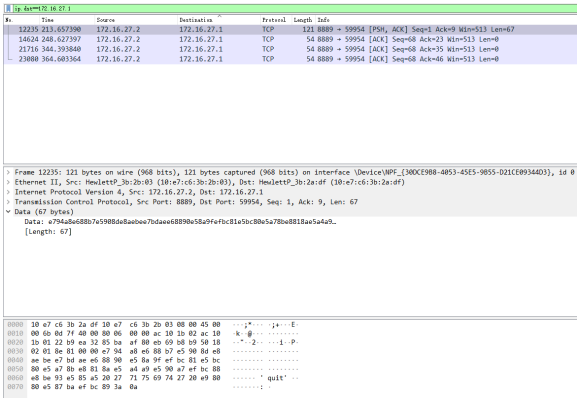
(b) 客户端聊天演示

图 13: 多客户端聊天演示

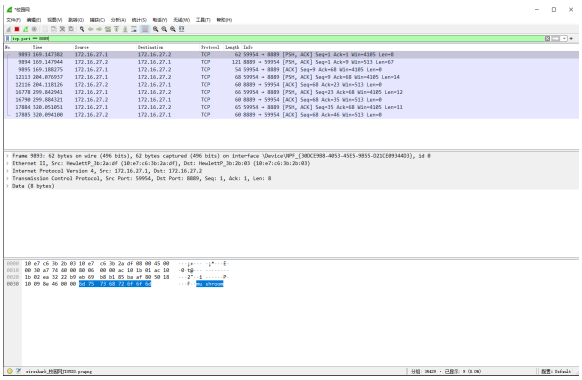
4.4 Wireshark 数据包分析

4.4.1 TCP 三次握手分析

通过 Wireshark 抓包，我们可以清晰地观察到 TCP 建立连接的三次握手过程：



(a) 服务器端抓包结果



(b) 客户端抓包结果

图 14: TCP 通信过程 Wireshark 抓包分析

表 4: TCP 三次握手过程分析

步骤	包类型	源端口	目标端口
1	SYN	客户端随机端口	8889
2	SYN-ACK	8889	客户端端口
3	ACK	客户端端口	8889

4.4.2 数据传输与连接断开过程分析

从 Wireshark 抓包结果可以看出，TCP 协议的完整通信过程包括：

- **连接建立：客户端发送 SYN 包请求连接，服务器响应 SYN-ACK 包，客户端确认 ACK 完成三次握手**
- **数据传输：聊天消息通过 PSH-ACK 包进行传输，每个数据包都有序列号和确认号**
- **连接断开：当客户端输入"quit" 命令时，执行四次挥手断开连接**
- **可靠性保证：接收方对每个数据包都进行 ACK 确认，确保数据传输的可靠性**

通过对比服务器端和客户端的抓包结果，我们可以验证 TCP 协议的双向通信机制和可靠传输特性，为理解网络协议工作原理提供了直观的实验证据。

参考文献

[1] Yi Xie. (2025). 中山大学计算机网络实验课程.

[2] WEB. (N.A.). 互联网内容.

[3] CSDN 博客. (2023). socket 技术详解. https://blog.csdn.net/fuhanghang/article/details/114528098